Get unlimited access to the best of Medium for less than $1/week. Become a member

Open in app ↗

Medium Search Write

LANGCHAIN

Lakshmi Pravallika Tatikonda 3 min read · Just now

1. Introduction to LangChain

LangChain is an open-source framework designed to help developers build applications powered by Large Language Models (LLMs) like OpenAI GPT or HuggingFace models.

In simple terms, LangChain helps us **connect LLMs with tools, memory, data sources, and workflows** to build intelligent applications.

Why LangChain is Important

Modern LLMs are powerful but have limitations:

- They cannot access real-time data directly
- They don't remember previous conversations well

- They cannot interact with external tools by default

LangChain solves these problems by providing a **modular framework** to build LLM-based systems.

Problems It Solves

- Prompt orchestration
- Multi-step reasoning (chains)
- Memory handling
- Tool and API integration
- Agent-based decision making

2. Core Components of LangChain

1. LLMs and Chat Models

LLMs are the core engines (like GPT-4, Claude, etc.).

Code Example

```
from langchain_openai import ChatOpenAI
```

```
llm = ChatOpenAI(model="gpt-4", temperature=0.7)

response = llm.invoke("Explain LangChain in simple terms")
print(response.content)
```

2. Prompt Templates

Used to structure input prompts dynamically.

Code Example

```
from langchain.prompts import PromptTemplate
```

```
template = PromptTemplate(
    input_variables=["topic"],
    template="Explain {topic} in simple terms for beginners"
)

print(template.format(topic="LangChain"))
```

3. Chains

Chains connect multiple steps together.

Code Example

```
from langchain.chains import LLMChain
from langchain_openai import ChatOpenAI
from langchain.prompts import PromptTemplate
```

```
llm = ChatOpenAI(model="gpt-4")

prompt = PromptTemplate.from_template("Explain {topic}")

chain = LLMChain(llm=llm, prompt=prompt)

result = chain.run("LangChain")
print(result)
```

4. Memory

Memory helps LLMs remember past conversations.

Code Example

```
from langchain.memory import ConversationBufferMemory
from langchain.chains import ConversationChain
from langchain_openai import ChatOpenAI
```

```
llm = ChatOpenAI(model="gpt-4")

memory = ConversationBufferMemory()

conversation = ConversationChain(
    llm=llm,
    memory=memory
)

print(conversation.predict(input="Hi, I am John"))
print(conversation.predict(input="What is my name?"))
```

5. Agents

Agents decide which tool to use dynamically.

Code Example

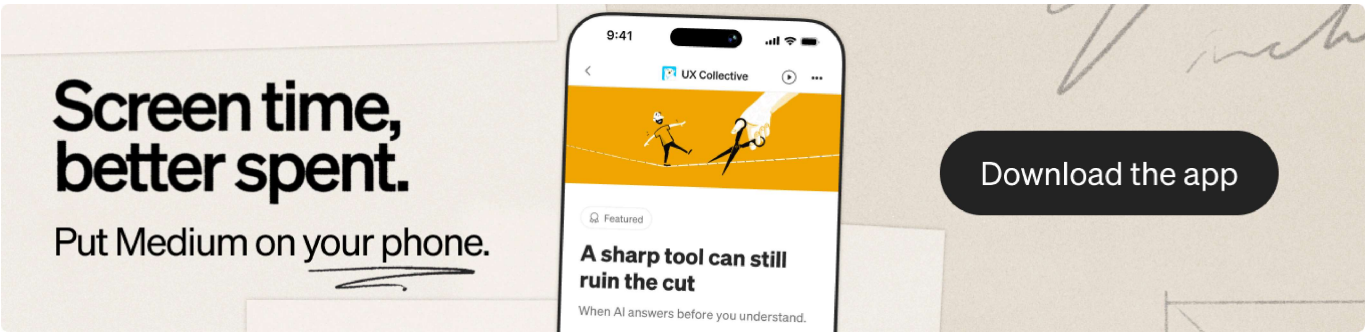
```
from langchain.agents import initialize_agent, Tool
from langchain_openai import ChatOpenAI
```

```
def calculator(x):
    return eval(x)
```

```
tools = [
    Tool(name="Calculator", func=calculator, description="Performs
math")
]

llm = ChatOpenAI(model="gpt-4")

agent = initialize_agent(tools, llm, agent="zero-shot-react-
description")

print(agent.run("What is 25 * 4?"))
```

6. Tools

Tools are external functions like search, calculators, APIs.



Example: Calculator, Search engine, database query system.

7. Document Loaders & Vector Stores

Used for RAG (Retrieval Augmented Generation).

Code Example

```
from langchain.document_loaders import TextLoader
```

```
loader = TextLoader("data.txt")
docs = loader.load()
```

```
print(docs[0].page_content)
```

Vector stores convert text into embeddings for search.

3. Architecture Explanation

Flow of LangChain System:

```
User Input
   ↓
Prompt Template
   ↓
LLM (GPT / HuggingFace)
   ↓
Chain (processing logic)
   ↓
Agent (optional decision making)
   ↓
Tools / APIs
   ↓
Final Output
```

Architecture Diagram (for submission)

You can draw this in diagram tool:

```
User
 ↓
Prompt Template
 ↓
LLM (GPT)
 ↓
Chain
 ↓
```

```
Memory / Agent
 ↓
Tools / APIs / Vector DB
 ↓
Response
```

4. Hands-on Code Examples

Basic LLM Call

```
from langchain_openai import ChatOpenAI
```

```
llm = ChatOpenAI(model="gpt-4")
print(llm.invoke("What is AI?").content)
```

PromptTemplate + Chain

```
from langchain.prompts import PromptTemplate
from langchain.chains import LLMChain
from langchain_openai import ChatOpenAI
```

```
llm = ChatOpenAI(model="gpt-4")

prompt = PromptTemplate.from_template("Explain {topic}")

chain = LLMChain(llm=llm, prompt=prompt)

print(chain.run("Machine Learning"))
```

Agent with Tool

```
from langchain.agents import initialize_agent, Tool
from langchain_openai import ChatOpenAI
```

```
def multiply(x):
    return str(eval(x))

tools = [
    Tool(name="Multiplier", func=multiply, description="Math
operations")
]

llm = ChatOpenAI(model="gpt-4")

agent = initialize_agent(tools, llm, agent="zero-shot-react-
description")

print(agent.run("What is 12 * 8?"))
```

Memory Example

```
from langchain.memory import ConversationBufferMemory
from langchain.chains import ConversationChain
from langchain_openai import ChatOpenAI
```

```
llm = ChatOpenAI(model="gpt-4")

memory = ConversationBufferMemory()

chat = ConversationChain(llm=llm, memory=memory)

print(chat.predict(input="My name is Alex"))
print(chat.predict(input="What is my name?"))
```

5. Real-World Use Cases

1. AI Chatbot for Customer Support

- Problem: Manual customer replies are slow
- Solution: LangChain chatbot with memory
- Components: LLM + Memory + PromptTemplate

2. Document Q&A System

- Problem: Searching large PDFs manually
- Solution: RAG system using vector DB
- Components: Document Loader + Embeddings + Retriever

3. AI Research Assistant

- Problem: Collecting data from multiple sources
- Solution: Agent-based system using tools
- Components: Agent + Tools + LLM

6. Advantages and Limitations

Advantages

- Modular architecture
- Easy integration with APIs
- Fast prototyping of AI apps
- Supports agents and tools

Limitations

- Higher latency due to multiple steps
- Debugging is complex
- API cost can be high
- Requires good prompt engineering

When NOT to use LangChain

- Simple chatbot apps
- Static rule-based systems
- Low-latency production systems

7. Conclusion

LangChain is a powerful framework for building LLM-powered applications. It helps developers connect models with tools, memory, and external data sources.

Key takeaways:

- Understand LLM orchestration
- Build modular AI pipelines
- Use agents and memory effectively

Future Scope:

- LangGraph for advanced workflows
- Multi-agent AI systems

- Autonomous AI applications

Langchain Python Programming Generative Ai Tools

Written by Lakshmi Pravallika Tatikonda

0 followers · 1 following

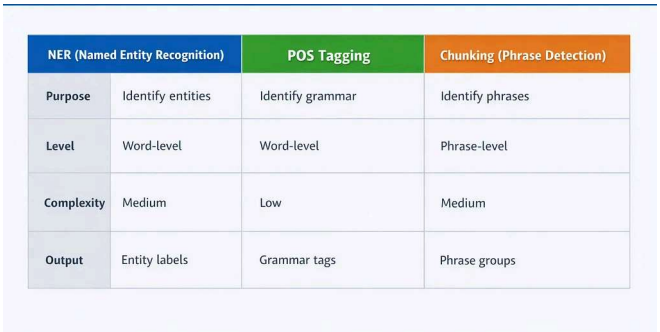
Edit profile

No responses yet

Lakshmi Pravallika Tatikonda

What are your thoughts?

More from Lakshmi Pravallika Tatikonda



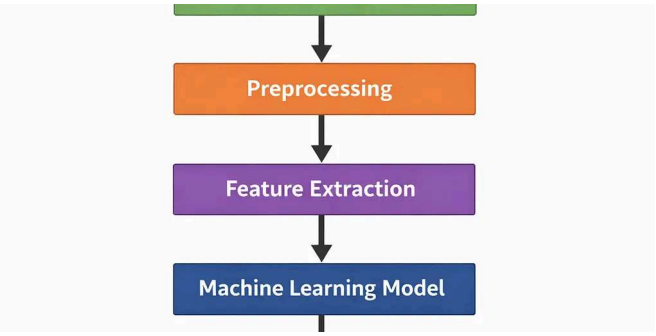
	NER (Named Entity Recognition)	POS Tagging	Chunking (Phrase Detection)
Purpose	Identify entities	Identify grammar	Identify phrases
Level	Word-level	Word-level	Phrase-level
Complexity	Medium	Low	Medium
Output	Entity labels	Grammar tags	Phrase groups

Lakshmi Pravallika Tatikonda

Understanding NLP Token Classification: NER, POS Tagging ...

Introduction to NLP & Token Classification

Apr 4 5



Lakshmi Pravallika Tatikonda

From Raw Text to Intelligence A Simple Guide to the NLP Pipeline

Introduction to NLP

Mar 29

Lakshmi Pravallika Tatikonda

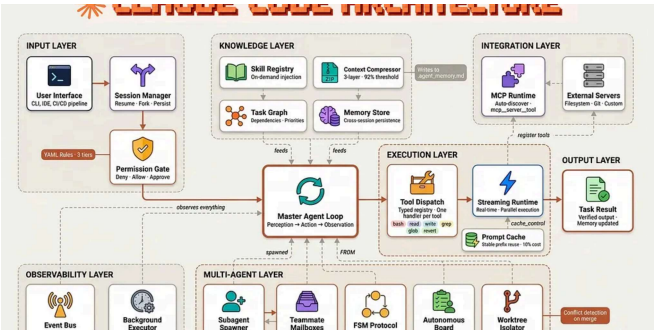
Choosing the Right Python Data Structure: A Beginner's Decision...

Introduction

Feb 16 1

See all from Lakshmi Pravallika Tatikonda

Recommended from Medium



In Level Up Coding by Fareed Khan

Building Claude Code with Harness Engineering

Multi-agents, MCP, skills system, context pipelines and more

6d ago 1K 8

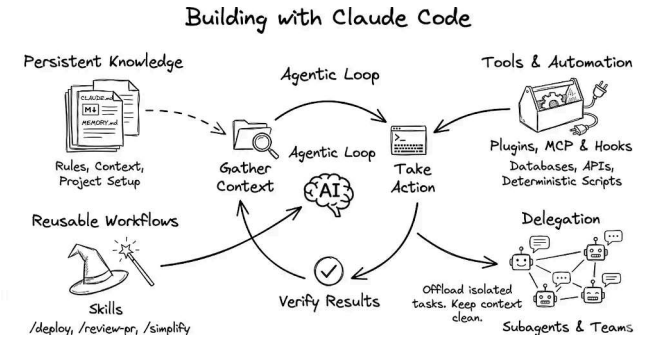
E2B	2.3B	4GB	85	6.1/10	6.5/10	On-device, edge
E4B	4.5B	8GB	38	7.4/10	7.8/10	Local dev, laptops
26B MoE	**3.8B active**	**18GB**	**42**	**9.1/10**	**8.9/10**	**Most use cases**

In Towards AI by Chew Loong Nian

I Tested All 4 Gemma 4 Models: The 26B One Is Cheating (In the...

Google dropped Gemma 4 this week under Apache 2.0 — free to use, free to...

Apr 4 153 1



In Data Science Collective by Farhad Malik

What I Learnt Using Claude Code to Build Production-Ready Apps

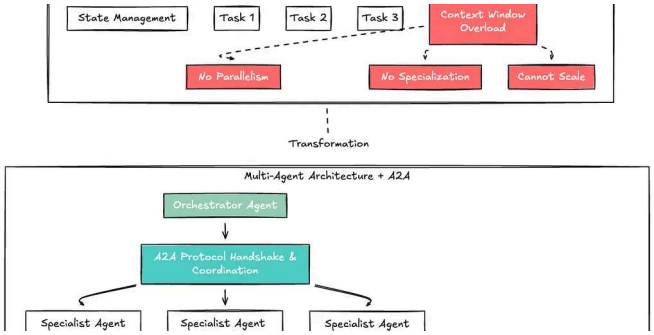
A practical guide covering the building blocks, hidden features, and tips that helped...

In AI Engineering BootCa... by Damilola Oyedunm...

A Beginner's Guide to Using MCP with LangGraph

If you've been following along with the LangGraph series, you already know how...

Mar 29 583 5



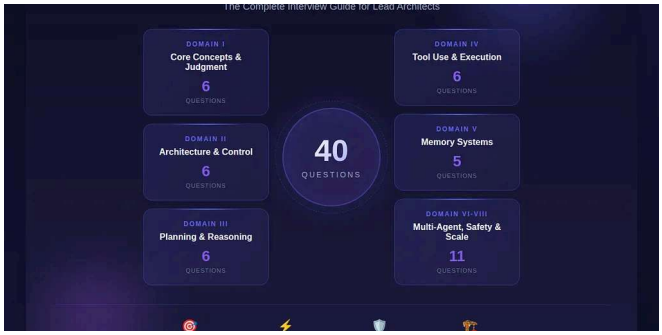
Yusuf Baykaloğlu

Multi-Agent Systems: Orchestrating AI Agents with A2...

A Deep Dive into Agent Architecture, Workflows, and Real-World Implementation

Jan 12 273 1

Nov 17, 2025 7 2



TechEon

The Complete Agentic AI System Design Interview Guide 2026

A senior engineer's handbook for navigating the toughest agentic AI interviews

Jan 29 107 2

See more recommendations